

PartnerAPI

Test Plan and Evidence

Software Engineering Project

October 6, 2026

Contents

1	Testing Objectives	2
2	Test Cases	2
3	Boundary Value Analysis	3
3.1	Request Rate	3
3.2	Room Count	3
4	Equivalence Classes	4
4.1	Room Count	4
5	Decision Table	4
6	Defect Density	4
7	Defect Removal Efficiency	5
8	Quality Gates	5
9	Test Evidence	5
10	Conclusion	6

1 Testing Objectives

Testing shall verify the functional and non-functional requirements of the PartnerAPI system.

The testing objectives are:

- Functional correctness
- Authentication
- Access control
- Rate limiting
- Hotel search
- Booking
- Cancellation
- API contract compliance
- Security
- Performance

2 Test Cases

The following test cases provide coverage of the major PartnerAPI functions and security controls.

Table 1: PartnerAPI Test Cases

ID	Test	Expected Result	Type
TC-001	Register partner	Partner is successfully registered	Functional
TC-002	Valid authentication	Request is accepted	Functional
TC-003	Invalid authentication	Request is rejected	Security
TC-004	Hotel search	Matching hotels are returned	Functional
TC-005	Create booking	Booking confirmation is returned	Functional
TC-006	Retrieve booking	Authorised booking is returned	Functional
TC-007	Cancel booking	Eligible booking is cancelled	Functional
TC-008	Revoke API key	Revoked key is rejected	Security
TC-009	Rate below limit	Request is accepted	Rate Limit
TC-010	Rate at limit	Request is handled according to policy	Boundary
TC-011	Rate above limit	HTTP 429 is returned	Boundary

ID	Test	Expected Result	Type
TC-012	Unauthorised partner access	Access is denied	Security
TC-013	Sandbox request	Sandbox response is returned	Integration
TC-014	Usage reporting	Usage information is recorded	Functional
TC-015	API documentation validation	Required endpoints are documented	Documentation

3 Boundary Value Analysis

Boundary Value Analysis (BVA) is used to test values at, below and above important input boundaries.

3.1 Request Rate

For an assumed configured limit of 100 requests per minute in the test configuration, the boundary cases are:

Table 2: Request Rate Boundary Tests

Input	Expected Result	Classification
99	Accepted	Below boundary
100	Accepted according to policy	Boundary
101	Rejected with HTTP 429	Above boundary

Note: The value above is a test configuration example for demonstrating the BVA method. Production limits shall be taken from the approved partner rate-limit configuration.

3.2 Room Count

The room-count boundary test shall use the approved API contract.

For a configured range of 1–10 rooms:

Table 3: Room Count Boundary Tests

Input	Expected Result
0	Reject
1	Accept
2	Accept
9	Accept
10	Accept
11	Reject

4 Equivalence Classes

Equivalence Class Partitioning divides possible inputs into groups that are expected to produce similar system behaviour.

4.1 Room Count

Table 4: Room Count Equivalence Classes

Class	Range	Expected Result
Invalid	< 1	Reject
Valid	1–10	Accept
Invalid	> 10	Reject

5 Decision Table

The following decision table tests partner authorisation, API-key validity and rate limiting.

Table 5: Partner Access and Rate-Limit Decision Table

Condition	R1	R2	R3	R4
Partner authorised	Y	Y	N	N
API key valid	Y	Y	Y	N
Rate limit exceeded	N	Y	N	N
Allow request	Y	N	N	N
Return HTTP 429	N	Y	N	N
Return authentication/access error	N	N	Y	Y

The table ensures that valid requests are allowed while requests that violate authentication, authorisation or rate-limit rules are rejected appropriately.

6 Defect Density

Defect density shall be calculated using:

$$\text{Defect Density} = \frac{\text{Number of Defects}}{\text{Software Size}}$$

The actual denominator shall use the approved project size measure.

For example, if the project uses thousands of lines of code (KLOC) as its size measure, the resulting metric may be reported as defects per KLOC.

7 Defect Removal Efficiency

Defect Removal Efficiency (DRE) is calculated as:

$$\text{DRE} = \frac{\text{Defects found before release}}{\text{Defects found before release} + \text{Defects found after release}} \times 100$$

Actual defect counts shall be taken from the project defect log.

A higher DRE indicates that a greater proportion of defects were identified before production release.

8 Quality Gates

The PartnerAPI release shall require the following quality gates:

- No unresolved critical security defects.
- Required functional tests passing.
- Rate-limit decision-table tests passing.
- Access-control tests passing.
- API error-rate target below 0.5%.
- Required documentation completed.

All quality gates should be reviewed before production activation of the first five partner agencies.

9 Test Evidence

Test evidence shall be maintained for the executed test cases and quality gates.

Evidence may include:

- Test execution results.
- API request and response records.
- Authentication and access-control results.
- Rate-limit test results.
- Security testing results.
- Defect records.

- Regression test results.
- Final quality-gate approval.

Actual execution results and defect counts shall be recorded in the project test evidence and defect log.

10 Conclusion

The PartnerAPI test plan provides coverage for functional behaviour, security, access control, rate limiting, boundary conditions, equivalence classes, decision-table scenarios and quality gates.

The defined test cases and quality criteria provide a structured basis for validating the API before production onboarding of the initial five partner agencies.